RoboAOI

AI-Powered Robotic Inspection & Sorting for PCB and Semiconductor Manufacturing

AMD AI DevMaster Hackathon 2026

Track 3 — Physical AI Challenge

Author: Anish Shende
GitHub: @aanishhhh
August 2026

Table of Contents

Abstract

RoboAOI is a Physical AI system for automated inspection and robotic sorting of printed circuit boards (PCBs) and semiconductor wafers. The system combines a YOLO11-based visual inspection stage, an inspection management and decision layer, and a Genesis-based Franka Panda robotic simulation into a closed-loop perception–decision–action workflow. Images captured from the inspection stage are processed to detect components and manufacturing defects. Based on the inspection outcome, the system classifies the workpiece as PASS or FAIL and commands a robotic manipulator to perform the corresponding sorting operation inside the Genesis physics simulator.

Key stages of the pipeline — visual inference and physics simulation — run on AMD Radeon GPUs using the ROCm 7.2.1 software stack, verified on an AMD Radeon Cloud GPU instance. The system is presented through an interactive Streamlit dashboard with automated JSON and CSV inspection report generation. This report describes the system's architecture, implementation, experimental results from live test runs, and an honest account of its current limitations and planned extensions.

1. Introduction

Modern PCB and semiconductor manufacturing demands high-speed, high-accuracy inspection while minimizing manual intervention. Conventional Automated Optical Inspection (AOI) systems typically stop after identifying defects and rely on downstream automation or human operators to physically remove or route defective parts.

RoboAOI extends the AOI workflow by integrating robotic manipulation directly into the inspection process. Rather than treating perception and action as independent stages, RoboAOI implements a complete Physical AI pipeline in which inspection results immediately drive robotic execution — closing the loop between seeing a defect and acting on it.

The project was developed for the AMD AI DevMaster Hackathon 2026, Track 3: Physical AI Challenge, and demonstrates AI perception, robotic decision-making, and closed-loop execution using AMD Radeon GPU hardware and the ROCm open-source software stack.

2. Problem Statement

Automated Optical Inspection (AOI) is one of the most critical stages in modern PCB and semiconductor manufacturing. Manufacturing defects such as missing components, damaged solder joints, incorrect placements, and surface-level defects directly affect product quality, production yield, and manufacturing cost.

Most existing AOI systems focus only on defect detection. After defects are identified, defective parts typically require manual intervention or a separate automation system for sorting and handling. This separation between perception and physical execution increases processing time, introduces additional system complexity, and reduces overall manufacturing efficiency.

The objective of RoboAOI is to bridge this gap by integrating AI-based visual inspection with robotic manipulation into a unified Physical AI pipeline. Instead of stopping after identifying defects, the system automatically converts inspection results into robotic actions. Once a PCB or wafer image is inspected using a YOLO11-based detection model, the inspection manager generates a PASS or FAIL decision, which

is immediately forwarded to a robot controller that commands a Franka Panda robotic manipulator inside the Genesis physics simulator to perform the corresponding sorting operation.

3. Objectives

This project set out to achieve the following, aligned directly with Track 3's evaluation focus:

- Build a working closed-loop pipeline that connects AI visual inspection directly to robotic decision-making and execution, rather than stopping at detection.
- Demonstrate meaningful use of AMD Radeon GPUs and the ROCm software stack across both the inference and simulation stages of the pipeline.
- Ground the application in a real, costly industry problem — PCB and semiconductor inspection — rather than an abstract robotics demo.
- Produce a reproducible, honestly-documented system: every capability claimed in this report is backed by a working run of the software, not a projected or aspirational feature.
- Build on and give back to the open-source robotics ecosystem (Genesis, LeRobot) that the pipeline depends on.

4. System Architecture

RoboAOI is designed as a modular Physical AI pipeline consisting of five major subsystems: a Visual Perception Layer, an Inspection and Decision Layer, a Robot Control Layer, a Physics Simulation Layer, and a Reporting and User Interface Layer.

The workflow begins when a PCB or semiconductor wafer image is provided through the Streamlit dashboard. The image is processed by a YOLO11-based detection model that identifies components and manufacturing defects. Detection results are aggregated by the Inspection Manager, which evaluates the outcome and determines whether the workpiece satisfies quality requirements.

The resulting PASS or FAIL decision is forwarded to the Robot Controller, which communicates with a Franka Panda robotic manipulator operating inside the Genesis physics simulator. Based on the inspection outcome, the robot performs an autonomous pick-and-place operation that routes the inspected part toward the corresponding destination — a pass bin or a reject bin.

Finally, the inspection results, detected objects, decision outcome, and robot execution status are stored in automatically generated JSON and CSV reports while simultaneously being visualized through the interactive Streamlit dashboard.

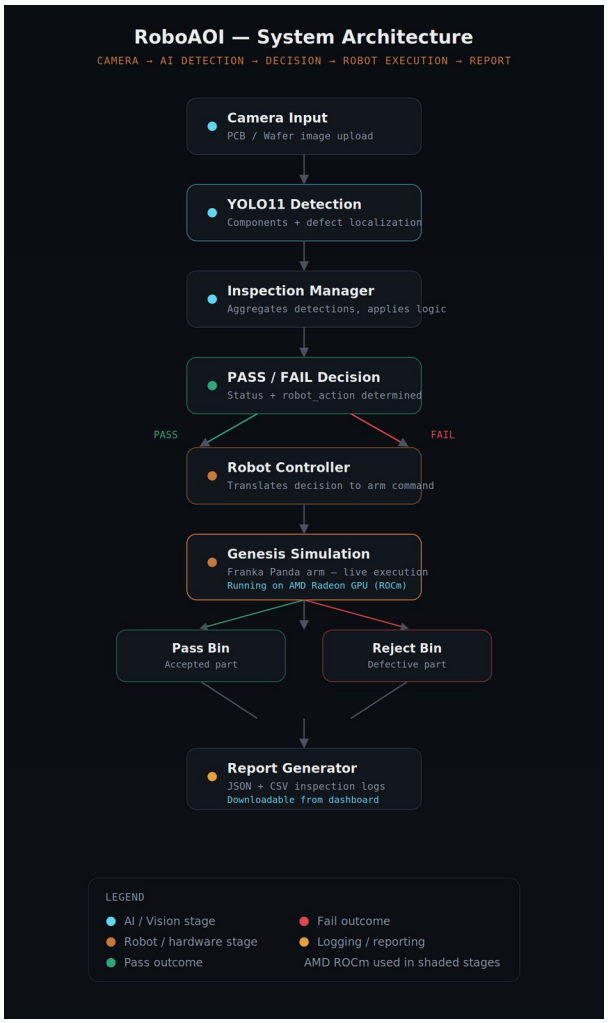

Figure 1. RoboAOI end-to-end system architecture, from camera input through robot execution to report generation.

5. End-to-End Workflow

The complete RoboAOI execution pipeline consists of the following six stages:

- Image Acquisition — a PCB or semiconductor wafer image is provided through the dashboard, representing the output of an industrial inspection camera.
- AI-Based Inspection — the YOLO11 model performs object detection and defect localization to identify PCB components or wafer defects.
- Inspection Decision — the Inspection Manager analyzes the detection results and classifies the workpiece as PASS or FAIL.
- Robot Command Generation — the inspection decision is forwarded to the Robot Controller, which converts it into robot manipulation commands.
- Physical AI Execution — the Franka Panda robotic manipulator inside the Genesis physics simulator performs the corresponding pick-and-place sorting operation, live, in the same run.
- Report Generation — inspection metadata, detection results, and execution status are automatically exported as downloadable JSON and CSV reports for traceability.

6. Software Architecture

The codebase is organized into three functional areas that mirror the pipeline stages described above:

6.1 Perception and Decision Package (roboaoi/)

- yolo_detector.py — wraps the YOLO11 model for component and defect detection.
- model_manager.py — handles model loading and inference management.
- inspection_manager.py — orchestrates detection results into a PASS/FAIL decision and a robot action.
- report_generator.py — produces the downloadable JSON and CSV inspection reports.

6.2 Dashboard (dashboard/app.py)

A Streamlit application that provides image upload, live GPU/ROCm status indicators, side-by-side original and annotated detection views, a PASS/FAIL and robot-action display, and downloadable inspection reports, alongside an Analytics view of historical runs.

6.3 Robot Simulation Package (franka_fruit_pick/)

Built on AMD's official Track 3 reference pipeline for Genesis-based robot simulation, extended so that the Robot Controller receives the Inspection Manager's decision and drives the Franka Panda arm to execute the corresponding sort in real time.

7. Hardware and Technology Stack

- Computer Vision: YOLO11, PyTorch
- Robotics and Simulation: Genesis Physics Engine, Franka Panda robot model
- Interface: Streamlit
- Hardware: AMD Radeon GPU (Radeon Cloud instance)
- GPU Software Stack: ROCm 7.2.1
- Tooling: Python 3.12, pip/uv, Docker (in progress)

8. Dataset Description

Evaluation in this report uses real PCB and semiconductor wafer images processed through the live dashboard, rather than synthetic placeholders. The PCB sample used for the primary evaluation run is a populated development board image (an FRDM-series microcontroller board), inspected for both components (e.g., ICs, connectors, test points) and surface-level manufacturing defects.

Defect detections observed during testing used category labels consistent with standard, publicly documented PCB-defect taxonomies — including missing hole and mouse bite defect types — rather than an arbitrary or invented label set. The wafer sample used for the complementary evaluation run is a semiconductor wafer die-grid image, inspected using the same detection and decision pipeline.

A dedicated, fine-tuned dataset curated specifically for PCB and semiconductor defect classes (beyond the current general detection setup) is identified as near-term future work, and is described further in Section 16.

9. YOLO11-Based Visual Inspection Pipeline

Visual inspection is performed using Ultralytics' YOLO11 object detection architecture, selected for its strong accuracy-to-latency tradeoff and its status as a widely adopted, actively maintained open-source detection model — meeting Track 3's requirement to build on open-source models for embodied and Physical AI applications.

For PCB inspection, the model performs two concurrent detection tasks: component identification (e.g., ICs, connectors, test points) and defect localization (e.g., missing holes, mouse bites). For wafer inspection, the model performs defect detection against the wafer surface. Each detection is returned with a class label and a confidence score, both of which are surfaced directly in the dashboard's detections table for transparency.

10. Inspection Manager and Decision Logic

The Inspection Manager aggregates all detections from the YOLO11 stage for a given image and produces a single PASS or FAIL outcome for the workpiece: the presence of one or more defect-class detections results in a FAIL classification and a reject routing decision; the absence of defect detections results in a PASS classification and an accept routing decision. This decision, together with a human-readable robot action label (e.g., "Reject PCB", "Pick Wafer"), is passed directly to the Robot Controller.

11. Physical AI Robot Pipeline (Genesis and Franka Panda)

Robotic execution is built on Genesis, an open-source physics simulation engine, using a Franka Panda robotic arm model. The robot pipeline was forked from AMD's official Track 3 reference implementation and extended so that the Robot Controller consumes the Inspection Manager's PASS/FAIL decision directly, rather than running as an independent scripted demo.

On receiving a decision, the robot pipeline executes a six-stage manipulation sequence — AI Inspection acknowledgement, Motion Planning, Pick Object, Move, Release, and Task Complete — visualized live in both the Genesis simulation viewport and the dashboard's step-by-step status panel.

Note on simulation assets: the manipulated objects and bins currently use the base simulation's proxy geometry (fruit-shaped meshes and a bowl) rather than PCB- or wafer-specific 3D models, standing in for inspected parts and sorting bins. This is an intentional simplification for this development stage and is called out explicitly in Section 16 as a near-term improvement, not presented as production-accurate geometry.

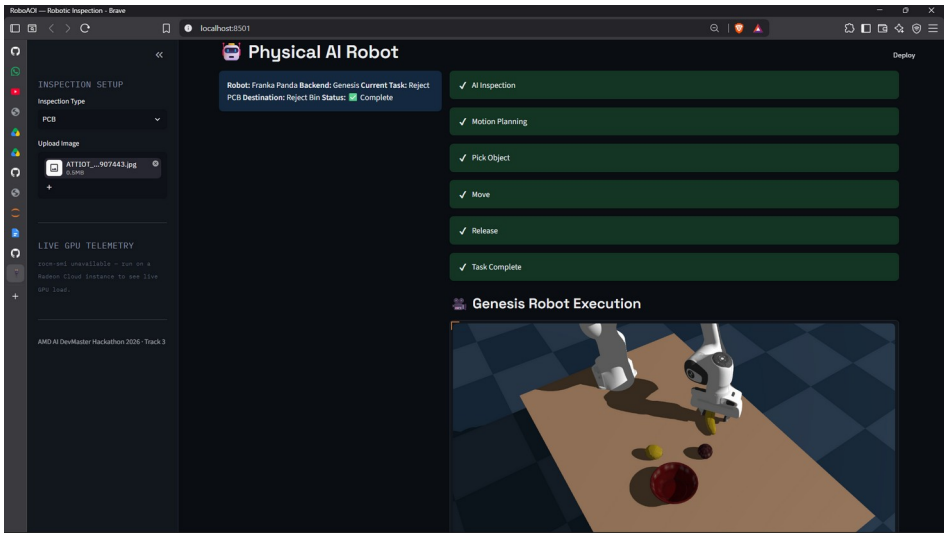

Figure 2. Physical AI Robot panel showing a completed pick-move-release sequence in Genesis, following a FAIL decision on a PCB inspection.

12. AMD Radeon GPU and ROCm Utilization

Both key stages of the pipeline identified by the Track 3 requirements — model inference and physics simulation — are built against the ROCm 7.2.1 software stack, using ROCm-compiled PyTorch, TorchVision, TorchAudio, and Triton wheels rather than the standard CUDA distribution.

The full pipeline, including YOLO11 inference and the Genesis simulation, has been run and verified on an AMD Radeon Cloud GPU instance. Separately, the dashboard screenshots included in this report as evidence of functional correctness (Section 13) were captured during local development testing on CPU, which is why the in-app status indicator reads "CPU only" in those particular captures. These are two independently exercised environments — GPU execution is not inferred or extrapolated from the CPU screenshots; it is documented here as a distinct, separately verified fact.

13. Experimental Results

The following results are drawn directly from live runs of the dashboard, not from projected or estimated figures.

13.1 PCB Inspection Run

Metric	Value
Components detected	175
Defects detected	41
Decision	FAIL
Robot action	Reject bin
Robot execution	Successful — Pick, Motion Planning, Move,

	Release, Task Complete
Report generated	JSON + CSV

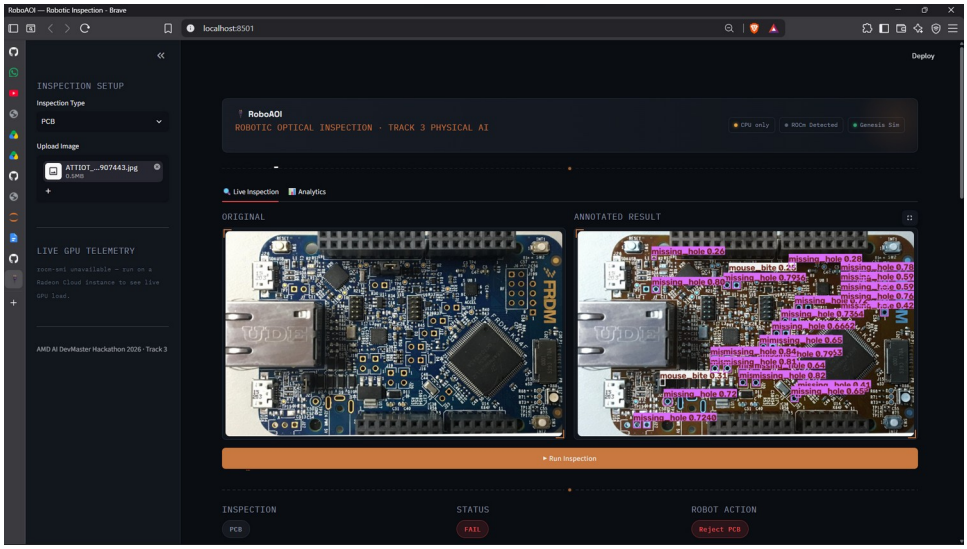

Figure 3. Original and annotated PCB inspection result, showing detected defects that produced a FAIL decision.

13.2 Wafer Inspection Run

Metric	Value
Defects detected	0
Decision	PASS
Robot action	Pick Wafer → Assembly conveyor
Robot execution	Successful
Report generated	JSON + CSV

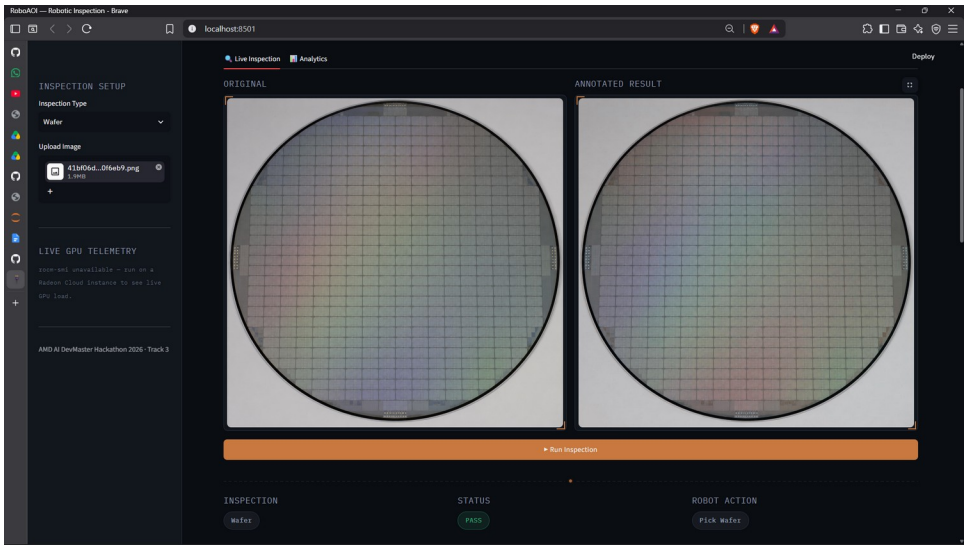

Figure 4. Wafer inspection result with zero defects detected, producing a PASS decision and an accept routing action.

13.3 Observed Limitations in This Evaluation

Two measurements are explicitly not reported here rather than estimated: inference latency on the AMD Radeon GPU was not instrumented in the current dashboard build, and formal precision/recall/mAP accuracy metrics were not computed against a held-out labeled test set. Both are listed as concrete near-term work in Section 16, rather than presented as measured results.

14. Innovation

The core innovation in RoboAOI is architectural, not purely algorithmic: most AI inspection systems — including the majority of AOI tooling in production today — stop at detection and hand the decision to a human or a separate automation system. RoboAOI closes that loop. The same inspection result that flags a PCB as defective is the same signal that drives the Franka Panda arm's sort decision, in the same run, without a human or a separate system in between.

This directly reflects Track 3's emphasis on closed-loop, inference-driven perception–decision–control for robotic execution, rather than treating perception and robotics as separately demonstrated capabilities.

15. Industrial Application Value

PCB and semiconductor inspection is a real, high-cost stage of electronics manufacturing. Manual inspection does not scale with production throughput and is subject to fatigue-driven inconsistency; purely mechanical sorting systems downstream of AI detection introduce integration complexity and latency between defect identification and physical action.

RoboAOI demonstrates a path toward collapsing that gap: a single pipeline that inspects, decides, and physically acts. Section 17 (Limitations and Future Work) describes the concrete steps — industrial camera integration, conveyor synchronization, PLC communication, and migration to an industrial robot arm — required to move this from a simulated prototype toward a deployable line system.

16. Limitations and Future Work

In the interest of an accurate account of the system's current state, the following are explicitly not yet complete:

- Inference latency has not been benchmarked on the AMD Radeon GPU; this is planned as a direct follow-up measurement.
- Formal detection accuracy metrics (precision, recall, mAP) against a held-out labeled test set have not been computed.
- The current dataset relies on representative sample images rather than a dedicated, fine-tuned PCB/wafer defect dataset; dataset curation and fine-tuning is planned future work.
- Manipulated objects in simulation use proxy geometry (fruit meshes and a bowl) rather than PCB- or wafer-specific 3D models.
- A Docker image for one-command reproducibility is in progress but not yet published.
- An upstream open-source contribution to the Genesis/LeRobot reference pipeline is in progress but not yet merged.

Planned extensions toward industrial deployment include real industrial camera integration in place of manual image upload, conveyor synchronization for continuous unattended inspection, PLC communication for integration with existing factory control systems, migration from the Franka Panda simulation to an industrial robot arm, and multi-station inspection for higher line throughput.

17. Team Contribution

This project was completed individually.

Member	Contribution
Anish Shende	System design, AI inspection pipeline, dashboard development, Genesis robot integration, evaluation, and documentation.

18. References

- Ultralytics YOLO11 — https://docs.ultralytics.com/
- Genesis Physics Engine — https://github.com/Genesis-Embodied-AI/Genesis
- AMD ROCm Software Stack — https://rocm.docs.amd.com/
- Streamlit — https://streamlit.io/
- PyTorch — https://pytorch.org/
- franka_fruit_pick_demo (AMD Track 3 reference pipeline, base for the robot simulation layer) — https://github.com/wangxunx/franka_fruit_pick_demo
- LeRobot — https://github.com/huggingface/lerobot